

CollegeCard

Complete Product + Learning Roadmap

Project idea: A digital college identity/business card. A student creates a profile, receives a unique profile URL and QR code, and anyone who scans the QR can view the student's public profile and connect through selected links.

Final goal: Build a real, mobile-friendly web application that can be tested by actual college students.

1. What You Are Building

Student	Action
1	Creates an account/profile
2	Enters name, college, branch, year, bio, skills and social links
3	Gets a unique profile URL, e.g. yoursite.com/u/vivek
4	Gets a QR code pointing to that URL
5	Shares the QR in college, events, resume, portfolio, etc.
6	Another person scans the QR and sees the public profile

2. Recommended Technology Stack

Area	Technology	Why
Frontend	HTML + CSS + JavaScript	Build the interface and make it interactive
Backend	Python + Flask	Handle accounts, profiles and requests
Database	SQLite first	Simple database for the MVP and learning
QR	Python QR library	Generate a QR containing the profile URL
Version control	Git + GitHub	Track and publish your code
Deployment	A suitable web host	Put the project online
Later	PostgreSQL / React / AI	Only after the MVP works

PHASE 0 — Define the MVP

Goal: Decide exactly what Version 1 does before writing code.

Profile fields: Photo, full name, college, branch, year, short bio, skills, GitHub, LinkedIn, Instagram, optional contact details.

V1 features: Create profile, edit profile, unique username, public profile, QR code, download/share QR, save contact.

Do NOT build yet: AI, chat, payments, complex analytics, mobile app, advanced admin systems, recommendation engines.

PHASE 1 — UI Design with HTML + CSS

Step 1: Project setup: Create index.html, profile.html, create-profile.html, qr.html, css/style.css, js/script.js and an images folder.

Step 2: Landing page: Build a simple homepage: navbar, value proposition, Create Profile button and a 3-step explanation: Create → Share → Connect.

Step 3: Profile page: Build a polished static student profile with photo, name, branch, college, bio, skills, social buttons, Save Contact and QR area. Use fake/hard-coded data first.

Step 4: Create Profile page: Build the form for name, college, branch, year, bio, skills and social URLs.

Skills: HTML structure, CSS, Flexbox, Grid, spacing, typography, buttons, cards and responsive design.

PHASE 2 — JavaScript Fundamentals

Step 5: Fundamentals: Learn variables, data types, operators, conditions, loops, functions, arrays and objects.

Step 6: DOM: Learn document.querySelector/getElementById, reading values, changing text, classes and HTML.

Step 7: Events: Learn click, submit, input and change events.

Step 8: Forms: Read user input, validate it, create a student object and display the profile dynamically.

Target: Clicking Create Profile should actually change the page instead of being a dead button.

PHASE 3 — Temporary Data with localStorage

Goal: Learn persistence before introducing a database.

Build: Save the student's profile object to localStorage, load it when the profile page opens, and support editing.

Architecture: Form → JavaScript → localStorage → Profile page.

Target: Close/reopen the browser and the profile still exists.

PHASE 4 — Python + Flask Backend

Python: Strengthen functions, modules, dictionaries/lists, exceptions and basic OOP as needed.

Flask: Learn routes, requests, responses, templates, static files and basic form handling.

Architecture: Browser → HTTP request → Flask/Python → data → response → browser.

Target: Move from a purely browser-based project to a real web application.

PHASE 5 — Database with SQLite

Goal: Store profiles permanently on the server.

Initial table: students: id, username, name, college, branch, year, bio, photo, github, linkedin, instagram, created_at.

Learn: Basic SQL: CREATE, INSERT, SELECT, UPDATE, DELETE and simple relationships when needed.

Target: A profile remains available even after the browser is closed and the server restarts.

PHASE 6 — Accounts and Dashboard

Build: Sign up, login, logout and a student dashboard.

Dashboard: My Profile, Edit Profile, Preview Profile, My QR and Logout.

Security: Never store raw passwords; use proper password hashing and secure session handling.

Target: Different students can manage their own profiles.

PHASE 7 — Unique Public Profile URLs

Build: Give each student a unique username/profile URL such as /u/vivek.

Backend logic: The route reads the username, retrieves the corresponding database record and renders the public profile.

Important: Keep the URL stable so the same QR can continue working when profile information changes.

PHASE 8 — QR Code System

Build: Generate a QR that contains only the public profile URL.

Correct design: QR → profile URL → Flask → database → current student profile.

Avoid: Do not encode all personal details directly into the QR. That makes updates and privacy harder.

Features: Generate, preview, download and share the QR.

PHASE 9 — Share + Save Contact

Share: Add a Share My Profile action using browser/device sharing where supported, with a copy-link fallback.

Save Contact: Generate a .vcf/vCard containing the student's chosen contact details.

Privacy: Make phone/email optional and allow students to decide what is public.

PHASE 10 — Profile Customization

Build: Allow profile photo, theme/accent, bio, skills and layout choices.

Possible themes: Classic, Minimal, Developer and Creative.

Rule: Keep the underlying profile data consistent even if the visual theme changes.

PHASE 11 — Projects and Portfolio

Build: Allow students to add projects with title, description, technologies, link and optional image.

Why: This turns the product from a simple contact card into a useful student identity/portfolio.

PHASE 12 — Mobile-First Responsiveness

Priority: Most QR scans will happen on phones, so the public profile must work extremely well on mobile.

Test widths: Around 360px, 390px, 414px, 768px, 1024px and desktop widths.

Check: Text size, buttons, image size, QR readability, scrolling, loading and touch targets.

PHASE 13 — Security + Privacy

Security: Password hashing, input validation, secure sessions, safe file uploads, URL validation and protection against common web vulnerabilities.

Privacy: Do not collect unnecessary information. Let users control whether phone/email/social links are public.

Account controls: Edit profile, change password and delete account/profile.

Verification: If you later show a 'verified student' badge, use a real verification mechanism; simply entering a college name is not verification.

PHASE 14 — Git, GitHub and Deployment

Git: Commit frequently with meaningful messages.

GitHub: Push the project, write a README, document setup and keep secrets out of the repository.

Deployment: Move from localhost to a real HTTPS website.

Learn: Environment variables, production configuration, domain setup, logs and basic deployment troubleshooting.

PHASE 15 — Test with Real Students

Start small: Give the product to about 5 students before trying to onboard an entire college.

Test tasks: Create profile, generate QR, scan on phone, edit profile, share link and test contact saving.

Observe: Do not explain everything immediately. Watch where users get confused; those are UX problems.

Fix: Prioritize broken flows and confusing screens over adding new features.

PHASE 16 — Version 2

Possible features: QR scan analytics, profile views, richer project portfolios, verified college identity, profile themes and better sharing.

Analytics rule: Collect only data you actually need and clearly explain tracking/analytics to users.

Verification: Potential methods include college email, college-issued ID verification or admin approval, depending on what the college supports.

PHASE 17 — AI Features

Only after the core product works: AI should enhance the product, not compensate for a weak foundation.

Ideas: AI bio generator, project-description generator, profile improvement suggestions and a profile assistant.

Example: Student enters rough details → AI suggests a concise professional bio → student reviews and approves it.

3. Complete Learning Roadmap

Stage	Learn	Use it in CollegeCard
1	HTML + CSS	Build landing, profile and form pages
2	JavaScript basics	Buttons, forms and dynamic UI
3	DOM + events	Make the profile creator interactive
4	Arrays + objects	Represent student/profile data
5	localStorage	Save profile data temporarily
6	Python fundamentals	Prepare for backend work
7	Flask	Routes, templates and requests
8	SQL + SQLite	Store student profiles
9	Authentication	Accounts and dashboards
10	QR generation	Create profile QR codes
11	Git + GitHub	Version control and collaboration
12	Deployment	Publish the real application
13	Security	Protect accounts and data
14	UX/testing	Test with real students
15	Advanced tools	React/PostgreSQL/AI only when justified

4. Recommended Project Structure

Start simple. The structure can evolve as the application grows.

```
collegecard/  
├─ index.html  
├─ profile.html  
├─ create-profile.html  
├─ qr.html  
├─ css/style.css  
├─ js/script.js  
├─ images/  
└─ README.md
```

When Flask is introduced, the structure will change to include templates/, static/, the Flask application file, database/model code and configuration.

5. Milestones — The Only Things You Need to Focus on Now

Milestone	Definition of Done
1. Static UI	A polished CollegeCard profile exists using only HTML + CSS.
2. Interactive UI	A user fills a form and JavaScript generates/updates their profile.
3. Persistent local prototype	Profile data survives browser refresh using localStorage.

Milestone	Definition of Done
4. Real web app	Flask + SQLite store and retrieve profiles.
5. Accounts	Students can sign up, log in and edit their own profile.
6. Public URLs	Every profile has a stable unique URL.
7. QR	Every profile has a QR pointing to its public URL.
8. Production MVP	The site is deployed and works on phones.
9. Real-user test	A small group of students can use it without you guiding every step.

6. What NOT to Do

- Do not start by learning every technology. Learn what the current milestone requires.
- Do not copy a giant full-stack tutorial and call the project finished if you cannot explain the code.
- Do not add AI just because the project is called a digital identity product.
- Do not collect phone numbers, addresses or other personal information unless there is a clear reason.
- Do not launch to hundreds or thousands of students before testing the basic flow with a handful of users.
- Do not spend weeks polishing animations while the profile creation and QR flow are still broken.

7. Your Immediate Next Step

Start with Milestone 1 only. Build the static profile UI: navbar → profile header → photo/name/branch/college → bio → skills → social buttons → QR area. Use fake data at first. No Python, no database, no authentication and no QR generation yet.

Once the static UI is solid, move to JavaScript and make the Create Profile form actually generate the profile. That is the correct starting point for your current skill level.